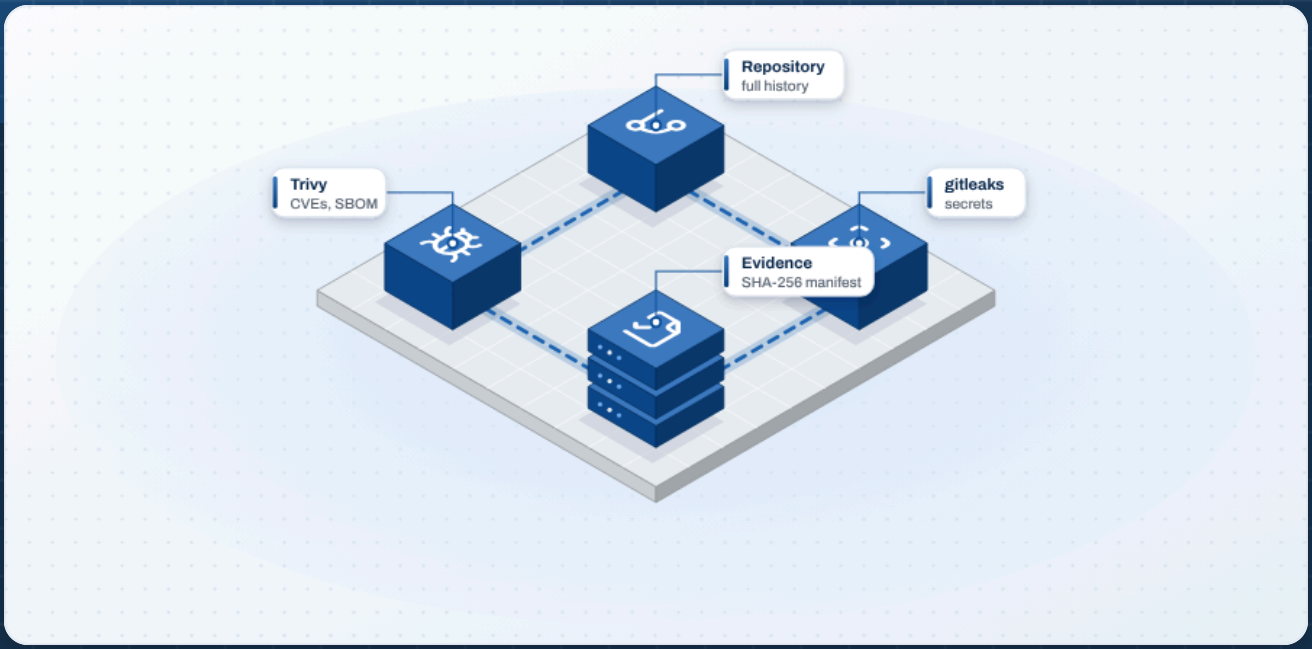
STANDARD OPERATING PROCEDURE · FIELD GUIDE SERIES

SOP-AZ-105

Scanning for Secrets and Supply-Chain Risk

gitleaks, Trivy and Dependabot across the full repository history, a licence inventory and a Key Vault posture check. Scans only; no code review.

DevSecOps | gitleaks | Trivy



DOCUMENT ID	SOP-AZ-105
VERSION	1.1
EFFECTIVE	October 2026
AUDIENCE	DevSecOps engineers, platform engineers, client approvers
AUTHOR	Richmond Dieujuste, Principal Consultant

Rich Dieu IT Solutions LLC · richdieu.com

Sanitized reference edition · client identifiers removed

Contents

1. Concepts You Need First 2

2. Access, Prerequisites and Planning 3

2.1 Level of access required 3

2.2 Prerequisites checklist 3

3. Procedure 4

4. Template · Scan Runner and Vault Check 7

5. Verification and Troubleshooting 8

5.1 Verify 8

5.2 Troubleshooting 8

6. Summary, Glossary and Review 9

6.1 Chapter summary 9

6.2 Glossary 9

6.3 Framework references 9

LEARNING OBJECTIVES

After working through this procedure you will be able to:

- Run secrets, vulnerability and licence scans across full history with written authorization.
- Triage findings without ever copying a secret value into a report.
- Confirm secrets live in a private vault the workload can only read.

CHAPTER 1

Concepts You Need First

A secret committed once stays in the repository history even after the line is deleted. Scanning only the current files misses it. Vulnerable dependencies and risky licences arrive the same quiet way, through a package someone added years ago.

KEY CONCEPT · THE ATTIC, NOT JUST THE LIVING ROOM

Cleaning the living room (the current branch) does not empty the attic (every old commit). gitleaks searches the attic; Trivy checks what was built from it; Dependabot watches what new boxes arrive.

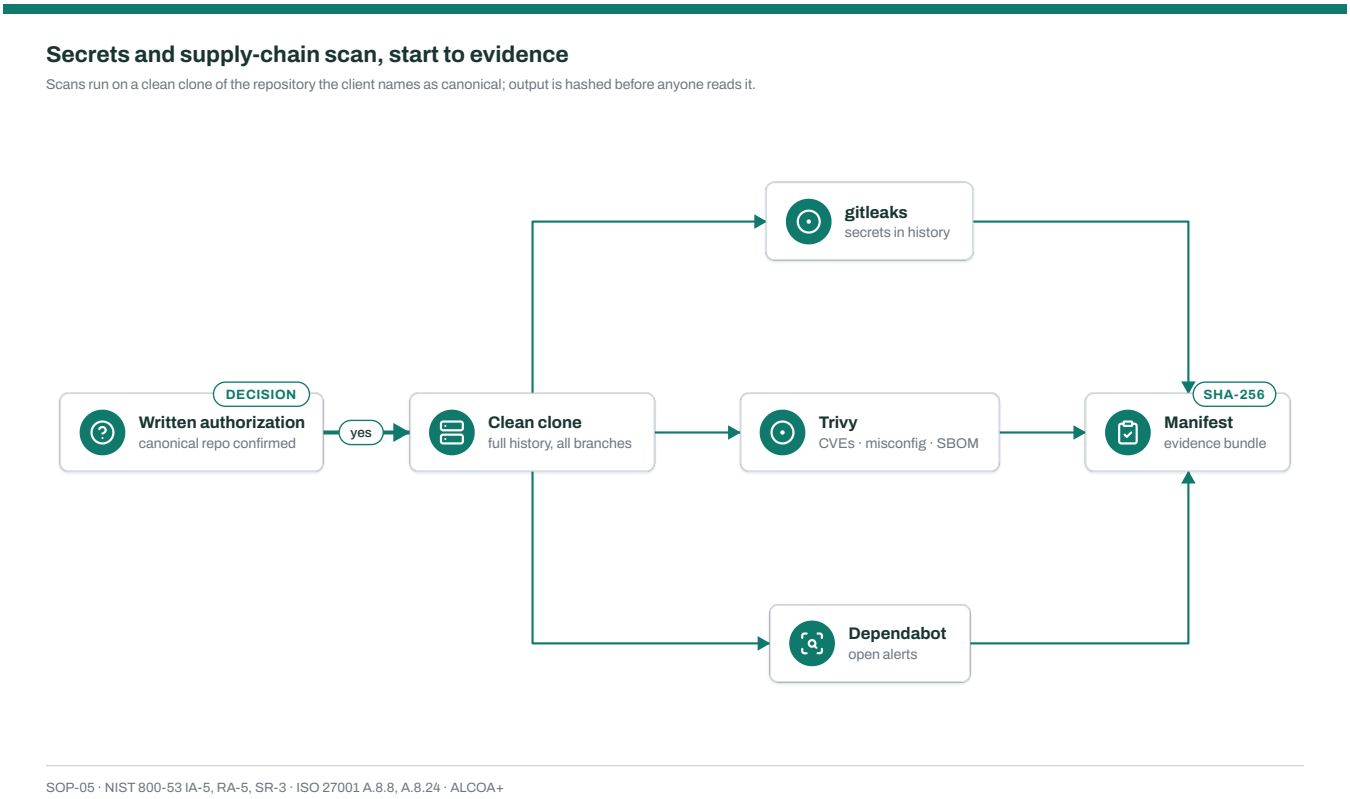


Figure 1. Authorization first, then a clean clone, three scans and a hashed manifest before anyone reads the results.

Tool	Finds	Output
gitleaks	Keys, tokens, passwords and connection strings in every commit	JSON with file, commit and rule (values redacted)
Trivy	CVEs, secrets and misconfiguration in files, IaC and images; SBOM	JSON, CycloneDX
Dependabot	Known-vulnerable dependencies, continuously	Open alerts from GitHub

CHAPTER 2

Access, Prerequisites and Planning

2.1 Level of access required

Task	Role	Notes
Scan the repository	Read on the canonical repository, all branches	Written authorization names the repo and dates
Export Dependabot alerts	Security events read	Token scoped to the one repository
Check the vault	Reader on the vault resource group	Data-plane read is not needed

CAUTION · AUTHORIZATION BEFORE SCANNING

Do not scan until you hold written authorization that names the repository the client treats as canonical. Never scan a fork or a developer copy.

2.2 Prerequisites checklist

- [ ] Written authorization on file.
- [ ] Canonical repository confirmed by its owner.
- [ ] gitleaks 8.x and Trivy 0.5x installed on the scanning machine.
- [ ] Evidence folder created with a run ID.

CHAPTER 3

Procedure

NOTE · HOW THESE SCREENS WERE CAPTURED

Screens in this chapter were captured read-only from a live engagement tenant. Client, product and people names, tenant, subscription and object IDs, email addresses and public IPs are replaced (`org`, `app`, `Contoso`, `00000000-…`, `203.0.113.10`). Compliance figures and unrelated rows are blurred. Portal labels move over time; the setting names are what matter.

STEP 1 Clone and scan history for secrets

1. Clone fresh with all branches: `git clone --mirror <repo-url>`.
2. Run `gitleaks git <repo> --log-opts="--all" --redact`.
3. Triage every hit: real secret (rotate in the vault, then purge history), test value, or false positive. Record file, commit and rule only.

CAUTION · NEVER PASTE A FOUND SECRET

A report that contains the secret has just leaked it again. Reference the commit and rule; rotate the secret before the report is sent.

STEP 2 Scan for CVEs, misconfiguration and licences

1. Run Trivy over the filesystem: `--scanners vuln,secret,misconfig --severity CRITICAL,HIGH`.
2. Scan the deployed image too, if there is one.
3. Generate a CycloneDX SBOM for the licence inventory and export open Dependabot alerts.

STEP 3 Check the vault is private

1. Open the vault › **Settings › Networking**. **Disable public access** should be selected.
2. Open **Private endpoint connections**; the stamp's endpoint should be Approved.

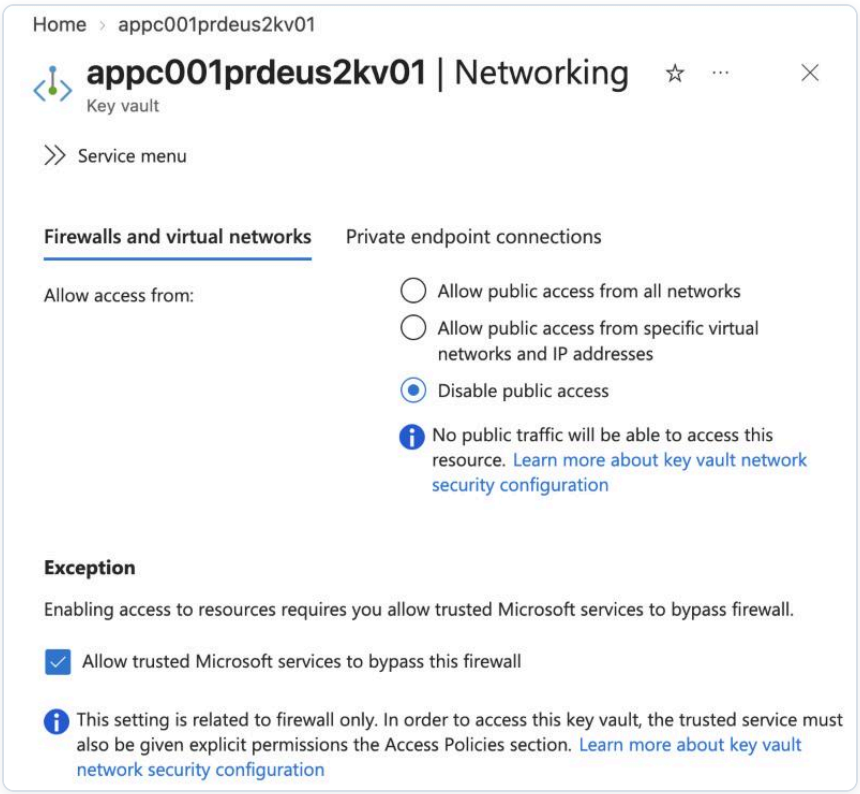


Figure 2. Vault networking: public access disabled; trusted Microsoft services may bypass for platform features only.

STEP 4 Prove it from outside

1. From a machine outside the private network open the vault › **Objects › Secrets**.
2. The portal should refuse to list secrets. That refusal is the evidence that a stolen portal session alone cannot read them.

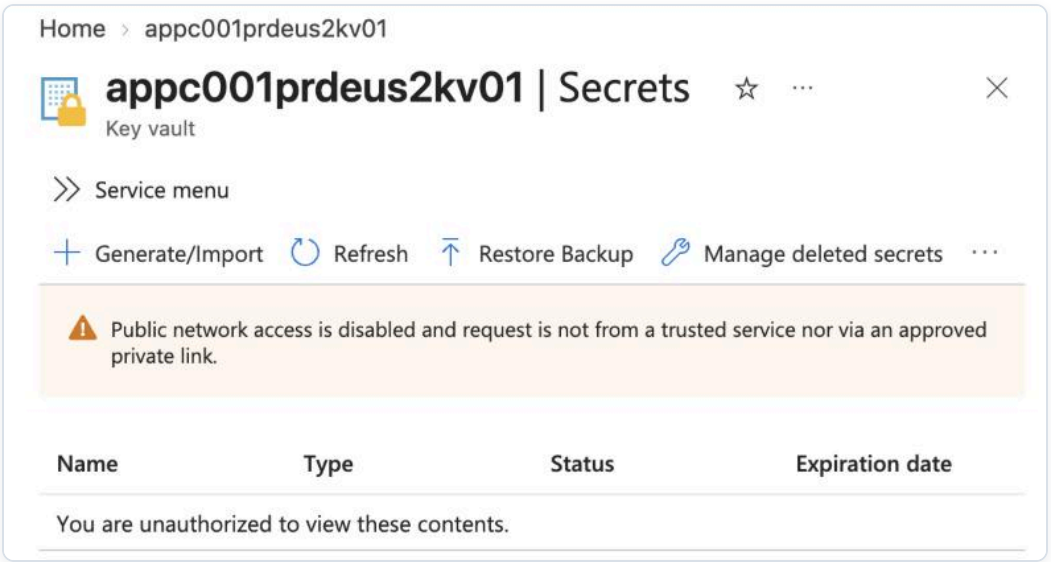


Figure 3. Listing secrets from outside the private network is refused by the vault firewall.

STEP 5 Hash and report

1. Hash every output into `MANIFEST.sha256` before anyone reads the results.
2. Report counts by severity, the triage result for each secret hit, and the fix path for each CVE (owner role and target date).

CHAPTER 4

Template · Scan Runner and Vault Check

SOP-05-SECRETS-SCAN.SH · SCAN RUNNER · GITLEAKS, TRIVY, SBOM, MANIFEST BASH

```bash
#!/usr/bin/env bash
# SOP-05 template: secrets and supply-chain scans. Scans only, no code review.
# Run in a clean clone of the repository the client names as canonical, with written authorization on file.
set -euo pipefail
REPO="${1:?path to repo clone}"; OUT="${2:-./scan-$(date -u +%Y%m%dT%H%M%SZ)}"; mkdir -p "$OUT"

# 1. gitleaks across the full history (every commit, every branch)
gitleaks git "$REPO" --log-opts="--all" --redact --report-format json --report-path "$OUT/gitleaks.json" || true

# 2. Trivy: vulnerabilities, secrets and misconfiguration in the filesystem and IaC
trivy fs "$REPO" --scanners vuln,secret,misconfig --severity CRITICAL,HIGH --format json -o "$OUT/trivy-fs.json"
#    and the container image actually deployed (if any)
# trivy image <registry>/<image>:<tag> --severity CRITICAL,HIGH --format json -o "$OUT/trivy-image.json"

# 3. Licence inventory (SBOM) for the open-source inventory deliverable
trivy fs "$REPO" --format cyclonedx -o "$OUT/sbom.cdx.json"

# 4. Dependabot alerts (GitHub; needs a token with security_events read)
# gh api "repos/<owner>/<repo>/dependabot/alerts?state=open&per_page=100" > "$OUT/dependabot-open.json"

# 5. Hash everything for the evidence bundle (ALCOA+: attributable, original, accurate)
( cd "$OUT" && sha256sum * > MANIFEST.sha256 )
echo "Findings: $(jq length "$OUT/gitleaks.json") gitleaks hits; criticals: $(jq '[.Results[]?.Vulnerabilities[]?
| select(.Severity=="CRITICAL")] | length' "$OUT/trivy-fs.json")"
```

SOP-05-KEYVAULT-CHECK.PS1 · VAULT POSTURE · READ ONLY POWERSHELL

```powershell
<#  SOP-05 template: Key Vault posture (read-only). Exit criterion: no plaintext secrets, vault private, VM read-
only. #>
param([Parameter(Mandatory)][string]$VaultName, [Parameter(Mandatory)][string]$ResourceGroup)
$kv = Get-AzKeyVault -VaultName $VaultName -ResourceGroupName $ResourceGroup
[pscustomobject]@{
  RbacMode          = $kv.EnableRbacAuthorization          # expect True
  PurgeProtection   = $kv.EnablePurgeProtection            # expect True
  PublicNetwork     = $kv.PublicNetworkAccess              # expect Disabled
  IpRules           = @($kv.NetworkAcls.IpAddressRanges).Count  # expect 0
}
Get-AzRoleAssignment -Scope $kv.ResourceId | Select-Object DisplayName, ObjectType, RoleDefinitionName
# Names only, never values
Get-AzKeyVaultSecret -VaultName $VaultName -ErrorAction SilentlyContinue | Select-Object Name, Enabled, Expires,
ContentType
```

CHAPTER 5

Verification and Troubleshooting

5.1 Verify

- [ ] Manifest hashes match the delivered files.
- [ ] Every gitleaks hit has a triage result; every real secret has a rotation record.
- [ ] No CRITICAL rows in Trivy, or each has an accepted risk ID.
- [ ] Vault check prints RBAC True, purge protection True, public access Disabled, IP rules 0.

5.2 Troubleshooting

Symptom	Likely cause and fix
gitleaks finds nothing in an old repo	Shallow clone. Re-clone with full history (`--mirror`).
Trivy DB download fails	Egress blocked. Use an offline DB bundle or run from an allowed network.
Vault check shows an IP rule left behind	A bootstrap window was not closed. Remove the rule and record it as a finding.

CHAPTER 6

Summary, Glossary and Review

6.1 Chapter summary

- Scan full history, not the current branch.
- Hash before reading; never copy a secret value.
- Secrets live in a private vault; the portal refusal from outside is the proof.

Check your understanding

1. Why does deleting a secret from the latest commit not remove it?
2. What must exist before any scan starts?
3. What three outputs make up the evidence bundle?

6.2 Glossary

SBOM	Software bill of materials: every component and licence in a build.
CVE	A publicly catalogued vulnerability.
Canonical repository	The repository the client names as the source of what is deployed.

6.3 Framework references

Framework	Controls
NIST SP 800-53 R5	IA-5(7), RA-5, SA-15, SR-3, SC-12
ISO/IEC 27001:2022	A.8.8, A.8.24, A.8.28
PCI DSS v4 (boundary)	6.3, 8.6